

A Self-Learning Utility-Based Agent for Adaptive Data-Pipeline Orchestration: An Environment–Optimiser–Reward Taxonomy of the Always-Execute Attractor

Ahmed Mamdough Mohamed Mohamed

Independent Researcher | ahmdmshazly@gmail.com

Abstract

Adaptive data-pipeline orchestration in cloud environments requires balancing pipeline value, compute cost, and failure risk under uncertainty. A natural design is a self-learning agent that maximises a single scalar utility $U = \alpha \text{ Value} - \beta \text{ Cost} - \gamma \text{ Risk}$. On a discrete-action simulator with one hundred jobs per episode, such an agent trained with policy gradient converges to a deterministic *always-execute* policy — it launches a ready task at every step and never scales, defers, or pauses. The tempting conclusion, drawn in an earlier version of this work, is that the *shape* of the scalar reward determines this trivial attractor. This paper shows that conclusion is wrong, and replaces it with a measured taxonomy. We hold the reward family fixed and vary, one at a time, the three factors a scalar-utility scheduler actually depends on: the environment, the optimiser, and the reward specification. Across five environments and two policy-gradient algorithms (REINFORCE and PPO), each with three initialisation seeds and fifty held-out evaluation seeds, always-execute appears for three *distinct* reasons. (i) On a benign environment, always-execute is the genuine optimum of the true objective — a Monte-Carlo policy-improvement probe shows no single non-execute deviation has positive advantage — so the learner is correct, not pathological. (ii) On environments where a non-execute action provably pays (contended capacity; load-dependent failure; heavy-tailed value), REINFORCE remains stuck at always-execute, but PPO escapes and beats it by +14 to +28 utility units ($p < 10^{-5}$), so the attractor is a property of the *weak optimiser*, not the reward. (iii) A defect in the original per-step reward — its risk term telescopes to a decaying counter rather than to the failed-job count, so the cumulative reward does not equal the reported utility — suppresses caution; correcting it changes a reckless policy (51% failure rate) into a careful one (5%). We further show, with an adequately powered test ($n=250$), that the learned policy is significantly, if slightly, worse than a trivial reflex baseline (-3.06 utility, $p = 2.3 \times 10^{-3}$), correct-

ing an earlier underpowered claim of equivalence ($p = 0.08$ at $n=50$, 28.8% power). The contribution is a reproducible diagnosis: a scalar-utility scheduler that “looks trivial” is explained by its environment, its optimiser, or a reward-specification bug — and a competent learner adapts, scaling where scaling pays, throttling where caution pays, and abstaining where neither does.

Keywords: reinforcement learning, data-pipeline orchestration, reward design, negative results, proximal policy optimization, reproducibility.

1. Introduction

Modern data pipelines run in cloud environments where workload intensity, resource availability, spot-price, and execution failures vary over time. In such settings, orchestration decisions cannot be reduced to static scheduling rules, because the same action can have very different consequences depending on congestion, price, contention, and recent failures. This makes adaptive pipeline orchestration a sequential decision problem rather than only a workflow-engineering one.

A practitioner reaching for a learned solution will naturally encode the competing objectives — complete valuable work, control compute cost, avoid unstable or failure-prone decisions — as a single scalar utility

$$U = \alpha \cdot \text{Value} - \beta \cdot \text{Cost} - \gamma \cdot \text{Risk}, \quad (1)$$

and train an agent to maximise it. This paper studies what such an agent learns, and why.

On a discrete-action, single-cluster orchestration simulator with one hundred jobs per episode and realistic stochastic events, an agent trained by policy gradient on Eq. (1) converges to a deterministic *always-execute* policy: at every decision step it launches a ready task, and it never scales the cluster, defers, reprioritises, or pauses. Five of its six actions receive near-zero probability. The same outcome survives an eighty-one-cell sweep of (α, β, γ) , an expansion of the observation from eight to four-

teen dimensions, and three network-initialisation seeds.

An earlier version of this work concluded from these invariances that the *shape* of the committed reward — not the algorithm and not the observation — determines the always-execute attractor, and framed it as a reward-shape-induced “trivial-attractor failure mode.” The present paper reports a skeptical re-examination of that claim and finds it does not hold. The earlier experiments varied three factors (reward weights, observation, seed) but held fixed the one factor that actually controls whether any non-execute action has positive expected value: the *environment*. They also rested on a per-step reward whose cumulative value does not equal the reported episode utility, and on an equivalence claim made at a sample size with 28.8% statistical power.

Contributions. We replace the single-cause claim with a measured, reproducible taxonomy.

1. **A reward-specification defect (§6.1).** The per-step reward used to train the agent telescopes, on its risk term, to a decaying failure counter rather than to the failed-job count used by the evaluation metric. Hence the cumulative reward is *not* the episode utility: the learner trains with an effectively zero failure penalty. We provide a config-gated correction for which the cumulative reward equals the utility exactly, verified by test.
2. **The benign environment has a trivial optimum (§6.3).** A common-random-number Monte-Carlo policy-improvement probe shows that on the committed environment no single non-execute deviation from always-execute has a positive-advantage confidence interval. Always-execute is the true optimum; the learner is correct.
3. **An adequately powered comparison (§6.4).** On a pre-registered, disjoint evaluation pool of 250 seeds, the learned policy is significantly — though slightly — *worse* than a trivial reflex baseline (-3.06 utility, $p = 2.3 \times 10^{-3}$), overturning an earlier underpowered claim of equivalence.
4. **The attractor is optimiser-determined, not reward-determined (§6.5).** On three environments engineered so that a non-execute action provably pays, REINFORCE remains at always-execute on 6/6 scaling-lever seeds, whereas PPO escapes on 5/6 and beats always-execute by $+14$ to $+28$ utility units ($p < 10^{-5}$). Under the identical reward and environment, a stronger optimiser removes the attractor. PPO

also *abstains* correctly: on the benign environment it stays at always-execute.

5. **The reward defect is material (§6.5).** On an environment where overload causes cascading failures, the original (buggy) reward leaves even PPO largely reckless ($51\% \rightarrow 44\%$ failure), while the corrected reward turns REINFORCE into a careful policy ($51\% \rightarrow 5\%$ failure, $+120$ utility).

The honest synthesis (§7) is that a scalar-utility scheduler which “looks trivial” is explained by the environment (the optimum really is trivial), the optimiser (a real but modest improvement the algorithm cannot find), or a reward-specification bug — never by “reward shape” as a standalone cause. We position this against four literatures (§2) and discuss why the constrained-MDP reformulation an earlier draft proposed as “the corrective” is mis-targeted (§7).

2. Related Work

This paper sits at the intersection of four literatures: learned schedulers for cloud and cluster systems, negative results and simple-baseline equivalence in deep reinforcement learning, reward design and reward hacking, and constrained Markov decision processes.

2.1. Learned Scheduling and Resource Management

The use of reinforcement learning for cluster and pipeline scheduling begins with DeepRM [1], which casts multi-resource packing as a policy-gradient problem and trains a REINFORCE agent against a negative-sojourn reward. Decima [2] extends the formulation to DAG-structured Spark workloads, coupling a graph-neural-network encoder over the ready frontier with an actor–critic policy and an input-dependent baseline [3], reducing average job completion time on TPC-H workloads. Non-RL approaches cover the same design space with other machinery: CherryPick [4] performs Bayesian optimisation over VM configurations under a runtime SLO; Resource Central [5] trains supervised predictors for downstream schedulers. A market- and optimisation-theoretic line expresses allocation as constrained convex or game-theoretic problems: Gavel [6] re-expresses scheduling policies as linear programs over an effective-throughput matrix; Pol-lux [7] maximises a concave goodput metric; Sia [8] and Shockwave [9] extend goodput to heterogeneous clusters and to dynamic-adaptation market equilibria. The contemporary LLM-serving lineage — vLLM [10], Sarathi-Serve [11], DistServe [12] — optimises SLO-constrained goodput under hard latency and memory budgets.

The divergence between our setting and this lineage is the *reward shape*. Each of DeepRM, Decima, Pollux, and the LLM-serving schedulers optimises an objective whose optimum is structurally non-trivial — through backpressure, an integrated completion-time penalty under stochastic arrivals, concavity, or hard constraints — so that “admit every ready job immediately” is not a local optimum. Our scalarised utility (1) lacks any analogous structural non-triviality. The present paper investigates exactly when that absence makes the optimum trivial, and finds that it is an environment property, not a property of the linear form.

2.2. Negative Results and Simple-Baseline Equivalence

A smaller, less-visible literature documents, rather than circumvents, reinforcement learning’s failure to beat simple baselines. Mania, Guy, and Recht [13] show that augmented random search over linear policies matches state-of-the-art deep RL on MuJoCo; Rajeswaran et al. [14] show the same for linear and RBF policies. Henderson et al. [15] and Engstrom et al. [16] document the reproducibility and implementation-attribution problems that make point-estimate comparisons unreliable; Andrychowicz et al. [17] find low-level design choices dominate algorithmic identity. Agarwal et al. [18] and Jordan et al. [19] supply the statistical machinery — stratified bootstrap, interquartile mean, probability of improvement, performance profiles — that makes null results visible rather than hidden behind a mean. We adopt that machinery directly (§6.4), and we emphasise the lesson it implies but that the present setting makes vivid: an apparent equivalence to a simple baseline can be an artifact of low statistical power, and a deep-RL “failure” can be a property of the optimiser rather than of the objective.

2.3. Reward Design and the Shape of the Objective

Ng, Harada, and Russell’s potential-based shaping theorem [20] establishes that adding a potential-difference term to a reward preserves the set of optimal policies. This is frequently glossed as “the shape of a reward determines the optimal policy set,” but the theorem concerns *additive potential-based shaping*, not the question of whether a given linear scalarisation of competing objectives has a trivial optimum on a given MDP; it therefore motivates, rather than entails, the present study. Amodei et al. [21] frame reward misspecification as a non-adversarial failure mode. Skalse et al. [22] define and characterise reward hacking and the conditions under which a reward simplification is “unhackable”; this constrains when a coarsen-

ing preserves a policy ordering but does not imply that linear scalarisations generically collapse to a dominating simplification. Pan et al. [23] give empirical evidence that reward “misweighting” produces phase transitions to degenerate behaviour; Knox et al. [24] provide sanity checks for reward design. The multi-objective RL literature [25], [26] argues that scalar reward is structurally insufficient for multi-attribute tasks and formalises Pareto and convex coverage sets as alternatives. The present paper is a minimally confounded case study within this body: we show that the apparent triviality is governed by the environment and the optimiser, isolate a concrete reward-specification bug, and decline to attribute the outcome to “reward shape” once those factors are varied.

2.4. Constrained MDPs as a Proposed Reformulation

A fixed linear scalarisation is the Lagrangian of a constrained MDP [27] at fixed dual multipliers: maximising $\mathbb{E}[V]$ subject to $\mathbb{E}[C] \leq c_{\max}$ and $\mathbb{E}[R] \leq r_{\max}$ has Lagrangian $\mathbb{E}[V] - \lambda_C \mathbb{E}[C] - \lambda_R \mathbb{E}[R]$, which at fixed λ is exactly Eq. (1). Reward-Constrained Policy Optimisation [28], the PID-Lagrangian method [29], Constrained Policy Optimisation [30], Lyapunov-based safe RL [31], and projection- and interior-point alternatives [32], [33] learn the multipliers instead of fixing them. An earlier draft proposed this reformulation as “the corrective” for the trivial attractor. We revisit that proposal in §7 and argue it is mistargeted: a CMDP cannot create a non-trivial optimum where the environment has none, and where the environment does provide a lever, a stronger optimiser or the corrected scalar reward already suffices.

3. The Orchestration Environment

The evaluation environment is a discrete-time, single-cluster simulator of a stochastic data-pipeline orchestration setting. An episode is a sequence of decision steps; at each step the agent observes a normalised state vector, selects one of six actions, and the world advances one tick. Every numeric parameter is read from a single configuration file; no constant is hard-coded in the agent or environment source. The simulator is deterministic given a seed: each seed spawns two independent `numpy` generators (one for the workload, one for the stochastic events) so that workload and event traces are reproducible independently.

3.1. State, Actions, Cost

State. The observation is a vector of eight real-valued features in $[0, 1]$ (Table 1); a fourteen-

Table 1: State vector $S \in [0, 1]^8$ (committed observation). A fourteen-dimensional variant appends five queue features and one spot-price forecast.

Feature	Meaning
ℓ_{cpu}	CPU load (in-use / capacity)
r_{ram}	free-RAM fraction
q	ready-queue depth (saturated at 20)
p_{spot}	current spot price, $[0.1, 1.0]$
n_{ready}	DAG-ready nodes (saturated at 20)
p_{job}	mean priority over active jobs
u_{deadline}	mean deadline urgency over active jobs
f_{recent}	recent-failure counter (saturated at 5)

Table 2: Action space. Parameters are fixed across a run.

Action	Effect
Execute_Ready_Job	launch the ready task maximising value $\times$ priority if it fits; else no-op
Defer_Job	one-step no-op; time advances
Scale_Up	+3 CPU, +4 RAM for three steps, then decays
Scale_Down	−2 CPU, −2 RAM, floored at the cluster minimum
Reprioritize_Queue	+0.05 priority to every queued job (cap 1.0)
Pause_LowPriority_Job	pause up to two running low-priority jobs

dimensional richer variant appends five per-job queue features and one spot-price forecast, used only where noted.

Actions. At each step the agent selects one of six named actions with fixed parameters (Table 2). Every action is legal from every reachable state; actions with no applicable target behave as no-ops.

Cost. The per-step cost is $\text{cost}(s, a) = p_{\text{spot}}(s)(w_{\text{cpu}} \text{cpu}_{\text{in use}} + w_{\text{ram}} \text{ram}_{\text{in use}})$ with $w_{\text{cpu}} = 0.6$, $w_{\text{ram}} = 0.4$, plus a per-action tariff that is zero in the committed configuration. Episode cost is the sum over steps.

3.2. Stochastic Processes and the Workload

Three exogenous processes drive non-determinism: a node-failure process, a data-spike process, and a bounded random walk on the spot price. Each has an explicit **mode** so that alternatives can be selected without code changes; this is the mechanism

by which we vary the environment in §6.5. Each episode instantiates $N = 100$ jobs by cloning one of three DAG templates (a linear chain, a fork–join, and a three-source aggregation) and perturbing durations and resource demands by discrete factors. Per-job value is drawn from $U(1, 5)$ in the committed (benign) workload; a heavy-tailed variant (§3.4) replaces a fraction of jobs with high-value “whales.”

3.3. The Shared Utility and Its Three Roles

The episode utility is

$$U = \alpha \sum_{\text{completed}} \text{Value} - \beta \text{TotalCost} - \gamma |\text{FailedJobs}|, \quad (2)$$

with committed weights $(\alpha, \beta, \gamma) = (1.0, 0.1, 1.0)$. The same scalar is intended to play three roles: the evaluation metric, the scoring term of the non-learning Utility-Based agent, and the reward signal for the learner. This “three-role identification” is the methodological instrument that, when intact, makes a learned/reflex equivalence attributable to the policy rather than to an objective mismatch. §6.1 shows the instrument was broken on the risk term, and supplies the fix.

3.4. The Environment Family

The central methodological move of this paper is to vary the environment while holding the reward family fixed. Table 3 lists the five environments. Each is a minimal, documented edit of the benign default; each either does or does not contain reachable states where a non-execute action has positive expected value, and we verify which by direct policy comparison (Table 8).

A note on failure semantics. The committed node-failure process selects at most one victim per step with probability 0.05, *independent of the action*. We verified that under this process — and under an independent per-running-task Bernoulli alternative at probabilities up to 0.12 — no throttling policy reduces expected failures by a significant margin, because expected failures are approximately p times total work and therefore policy-invariant. The committed “risk” axis is thus inert as a decision lever. The **cascade** mode is introduced precisely to create a failure process where caution *does* pay (keeping load below the threshold lowers expected failures), so that the reward’s role can be tested.

Table 3: The environment family. “Lever” = does any non-execute action have positive expected value? Verified empirically in Table 8.

Environment	Edit relative to benign / lever
benign	committed default; no exploitable non-execute action
tight	base capacity lowered to 6/8; all tasks still fit, but scaling raises parallelism. <i>Lever: Scale_Up</i>
heavytail	10% of jobs are whales (value 50) under tight capacity; richer 14-d state. <i>Lever: scale to finish whales</i>
cascade	failure probability jumps from 0.01 to 0.40 when CPU load > 0.4. <i>Lever: throttle/scale to avoid overload</i>
cascade-broken	cascade but trained with the original (uncorrected) reward. <i>Lever present but reward hides it</i>

4. Agents and Learning Algorithms

We study three agent designs and two learning algorithms.

Reflex Agent. A deterministic rule-based scheduler: execute if a ready task fits; scale up under queue pressure at moderate price; scale down when idle and over-provisioned at high price; otherwise defer. It does not read the utility and is weight-independent.

Utility-Based Agent. A non-learning agent that scores all six actions with a hand-designed rule plus control-flow guards and selects the maximum. An ablation (§6.2) shows its behaviour is dominated by a single binary “force-execute” gate; β and γ enter only through the sign of one score.

Self-Learning Agent (REINFORCE). A feed-forward policy (64×64 , tanh, softmax over six actions) trained by REINFORCE [34] with a sequence-specific per-timestep baseline in the style of DeepRM/Decima [1], [3], a three-stage curriculum, memoryless termination, and entropy regularisation. No value head is trained; the sequence baseline is the sole variance-reduction mechanism.

Self-Learning Agent (PPO). To test whether the always-execute attractor is a property of the *optimiser*, we add Proximal Policy Optimisation [35] with the *same* actor network, a separate value critic, generalised advantage estimation [36], the clipped surrogate objective, and ten epochs of minibatch updates per batch (Algorithm 1). Because

Algorithm 1 PPO trainer (actor shared with REINFORCE)

```

1: init actor  $\pi_\theta$ , critic  $V_\phi$  (both  $64 \times 64$ , tanh)
2: for iteration = 1, ..., 150 do
3:   collect  $B=16$  episodes under  $\pi_\theta$  on random train seeds
4:    $\hat{A}_t \leftarrow \text{GAE}(\lambda=0.95, \gamma=0.99)$  via  $V_\phi$ ;  $\hat{R}_t \leftarrow \hat{A}_t + V_\phi(s_t)$ 
5:   normalise  $\hat{A}$ 
6:   for epoch = 1, ..., 10; minibatch  $m$  do
7:      $\rho \leftarrow \exp(\log \pi_\theta(a|s) - \log \pi_{\theta_{\text{old}}}(a|s))$ 
8:      $L^{\text{clip}} \leftarrow \mathbb{E}_m[\min(\rho \hat{A}, \text{clip}(\rho, 1 \pm 0.2) \hat{A})]$ 
9:      $L \leftarrow -L^{\text{clip}} + 0.5 \mathbb{E}_m[(V_\phi - \hat{R})^2] - 0.01 \mathcal{H}[\pi_\theta]$ 
10:    Adam step on  $L$ ; clip grad-norm 0.5
11:  end for
12:  checkpoint best-by-validation policy
13: end for

```

the actor is identical to the REINFORCE policy, a PPO checkpoint is evaluated by the same greedy-argmax harness; the value critic is training-only. PPO differs from our REINFORCE only in its variance reduction (a learned critic and GAE in place of the Monte-Carlo sequence baseline) and in its multi-epoch clipped updates — the two ingredients that, in this study, separate finding a modest improvement from missing it.

Evaluation protocol. All learned policies are evaluated by deterministic argmax (no sampling). Seeds are partitioned into disjoint pools — training, validation (for best-by-checkpoint selection), and a held-out test pool — enforced at configuration load. Unless stated otherwise, evaluation uses the 50-seed held-out pool 200–249; the power analysis of §6.4 uses a separate pre-registered pool 1000–1249.

5. Diagnostic Methodology

Three diagnostics, each a committed script with a CSV artifact, turn the qualitative claims of the earlier draft into measurements.

Reward-identity check. For an identical always-execute trajectory we compute the episode utility (Eq. (2)) and the sum of the per-step training rewards, and decompose the difference by term. If the cumulative reward equals the utility, the three-role identification is intact; otherwise the learner optimises a different objective than the one it is scored against (§6.1).

Local-optimality probe. To decide whether always-execute is the true optimum on a given environment, we estimate the one-step policy-improvement advantage $A_\pi(s, a) = \mathbb{E}_{\text{future}}[Q_\pi(s, a) - V_\pi(s)]$ against the

always-execute policy π : at each visited state, deviate to a non-execute action once, then follow π to episode end, and score on the *true* metric. We Monte-Carlo the future with K paired continuation seeds to integrate out the exogenous-shock noise that breaks naive common-random-number pairing (different first actions consume different numbers of random draws), and we report per-action means with bootstrap confidence intervals rather than a maximum over actions, which is positively biased under noise (§6.3).

Power and reliable-style reporting. We report interquartile means with stratified-bootstrap confidence intervals, paired probability of improvement, performance profiles, two one-sided equivalence tests (TOST), and a paired- t power analysis, following [18]. Every new experiment’s evaluation pool and prediction are registered before the run.

6. Results

6.1. A Reward-Specification Defect

The per-step reward used to train the learner is $r_t = \alpha \Delta \text{Value}_t - \beta \text{cost}(s_t, a_t) - \gamma \Delta \text{Risk}_t$, and an earlier draft asserted that $\sum_t r_t$ equals the episode utility, so that “the agent is trained on exactly the objective it is scored against.” This is false on the risk term. Let R_t denote the normalised recent-failure counter, which is incremented on each failure and decremented by one each step. The reward defines $\Delta \text{Risk}_t = R_t - R_{t-1}$, so the cumulative reward telescopes to

$$\sum_t r_t = \alpha \text{CompletedValue} - \beta \text{TotalCost} - \gamma (R_T - R_0) = U + \gamma (|\text{FailedJobs}| - R_T), \quad (3)$$

using $R_0 = 0$ and Eq. (2). Because R_T decays to ≈ 0 by episode end while $|\text{FailedJobs}|$ is a monotone count (≈ 13 – 23 per episode), the gap $\gamma(|\text{FailedJobs}| - R_T)$ is large and positive. Table 4 measures it on ten seeds: the value and cost terms telescope exactly, but the cumulative reward exceeds the utility by a mean of $+15.78$, matching Eq. (3) to machine precision.

The consequence is that every policy trained with this reward optimises $\alpha \text{Value} - \beta \text{Cost}$ with the failure penalty silently near zero, while being *scored* with the full penalty. We supply a config-gated correction, $\Delta \text{Risk}_t = (\text{jobs newly failed at step } t)$, for which $\sum_t r_t = U$ exactly, pinned by a regression test. On the benign environment, where failures are decision-independent (§3.4), the correction does not change the always-execute outcome; §6.5 shows it is decisive where caution actually pays.

Table 4: Reward-identity check (seeds 200–209, always-execute trajectory). The cumulative training reward does not equal the episode utility: the risk term telescopes to a decaying counter, not to the failed-job count.

Quantity	Result
$\text{mean}(\sum_t r_t - U)$	$+15.78$
$\max \sum_t r_t - U $	23.0
$\sum_t r_t - U =$	exact ($< 10^{-9}$)
$\gamma(\text{Failed} - \text{counter}_{\text{final}})?$	
value term telescopes to completed value	yes
cost term telescopes to total cost	yes
risk term telescopes to $ \text{FailedJobs} $	no

Table 5: Reflex vs. non-learning Utility-Based agent, 50 matched seeds, committed weights. Negative Δ means Utility-Based underperforms Reflex; for cost and failure rate a negative Δ is an improvement.

Metric	Reflex	Utility	Δ (p)
Total utility	121.0	111.5	-9.5 (0.025)
Completion rate	0.637	0.524	-0.11 (10^{-8})
Value-wtd. completion	0.697	0.596	-0.10 (10^{-12})
Total completed value	210.0	179.6	-30.4 (10^{-12})
Failure rate	0.145	0.126	-0.019 (10^{-3})
Total compute cost	744.5	554.2	-190 (10^{-8})

6.2. The Non-Learning Agent and Ceremonial Coefficients

Under the committed weights and 50 matched seeds, the hand-designed Utility-Based Agent *underperforms* the Reflex baseline on its own objective (Table 5): total utility 111.5 vs 121.0 ($\Delta = -9.47$, $p = 0.025$), with lower completion and lower completed value, but also lower cost and failure rate. An eighty-one-cell sweep of (α, β, γ) finds no cell that beats Reflex on the shared utility; the grid-best cell trails by 5.2 units. An action-trace ablation explains why: zeroing β or γ changes 0 of 6000 per-step decisions, and 46 of 81 grid cells collapse into a single behavioural class. The agent’s behaviour is set by a binary force-execute gate, not by the utility it appears to maximise. This motivates a learned policy — but, as the rest of this section shows, “learned” must be qualified by *which* learner.

6.3. The Benign Environment Has a Trivial Optimum

The learner, trained on the benign environment, converges to a deterministic always-execute policy: on all 50 held-out seeds it emits `Execute_Ready_Job` at every one of 15,000 steps.

Table 6: Local-optimality probe on the benign environment: one-step policy-improvement advantage of each non-execute action vs. always-execute (true metric, 16 paired futures, 75 states). No interval clears zero.

Action	mean adv.	95% CI
Defer_Job	−1.38	[−2.13, −0.68]
Scale_Up	−1.38	[−2.13, −0.67]
Scale_Down	−7.18	[−8.40, −6.01]
Reprioritize_Queue	−0.64	[−1.43, +0.20]
Pause_LowPriority_Job	−2.63	[−3.55, −1.67]

Is this a failure? The local-optimality probe answers no. Table 6 reports the Monte-Carlo one-step-improvement advantage of each non-execute action against always-execute, scored on the true metric, over 75 probed states with 16 paired futures each. *No* non-execute action has a confidence interval above zero; every one is, on average, a negative-advantage deviation. Always-execute is a local optimum of the true objective on this environment. (An earlier, single-future variant of this probe reported a misleading “70% of states have an improving deviation,” an artifact of order-statistic bias from maximising over five actions and of event-RNG desynchronisation; the corrected probe removes both.)

The reason is structural. All one hundred jobs exist at episode start; every task fits on the minimum cluster, so scaling is never required for feasibility; value is light-tailed, so no job is worth protecting; and failures are decision-independent, so no action reduces risk except not running, which forfeits value. On such an environment the scalar utility’s optimum genuinely is always-execute, and a learner that finds it is correct. Attributing the outcome to “reward shape” confounds the reward with a benign environment.

6.4. An Adequately Powered Comparison with the Reflex Baseline

An earlier draft reported that the learned policy “matches the Reflex baseline to within sampling noise” at $p = 0.08$ on 50 held-out seeds. Two corrections are required. First, evaluation is deterministic argmax, so there is no sampling at evaluation; the variation across seeds is workload variation, and the learned and reflex policies are two *different* deterministic policies (the learner emits **Execute** unconditionally and no-ops in capacity-blocked states, whereas Reflex scales or defers there; they differ in realised effect on 6.4% of steps). Second, $p = 0.08$ at $n = 50$ is an underpowered failure to reject: the paired effect size is $d_z = -0.20$, at which the test has 28.8% power.

Table 7: Learned policy (REINFORCE, argmax) vs. Reflex on a pre-registered disjoint pool, $n = 250$. reliable-style estimates with stratified-bootstrap CIs.

Quantity	Learned	Reflex
mean utility	125.5	128.5
IQM utility	124.9	129.4
paired difference (Learned − Reflex)		
mean Δ	−3.06 [−4.94, −1.19]	
Wilcoxon p	2.3×10^{-3}	
$P(\text{Learned} > \text{Reflex})$	0.456 [0.404, 0.508]	
power at $n=50$ / $n=250$	0.288 / 0.889	
TOST equivalent at ± 3 / ± 5	no / yes	

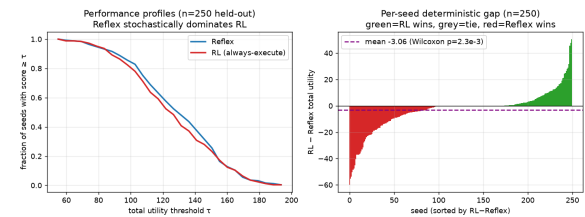


Figure 1: Left: performance profiles on the $n = 250$ pool — Reflex (blue) stochastically dominates the learned always-execute policy (red). Right: per-seed Learned − Reflex utility (green = learner wins, grey = exact tie, red = Reflex wins); mean -3.06 , $p = 2.3 \times 10^{-3}$.

We therefore re-evaluate on a pre-registered, disjoint pool of 250 seeds (1000–1249), with the existing checkpoint and greedy argmax (Table 7). The learned policy is significantly — though slightly — *worse* than Reflex: a paired difference of -3.06 utility units, $p = 2.3 \times 10^{-3}$, with the Reflex agent winning on 100 of 250 seeds and the learner on 78 (the remainder tie exactly). Probability of improvement is 0.456. The gap is small enough to be TOST-equivalent within a ± 5 -unit margin but not within ± 3 . The honest statement is therefore “significantly, if slightly, worse than the reflex baseline,” not “equivalent.” Figure 1 shows the performance profiles — Reflex stochastically dominates the learner — and the per-seed gap.

6.5. The Taxonomy: When Always-Execute Appears, and Why

We now vary the environment so that a non-execute action provably pays, and ask two questions of each environment: does the lever exist (Table 8), and do the two learners find it (Table 9, Figure 2)?

Four findings follow.

(i) Benign: every competent agent stays trivial. On the benign environment, PPO converges to $\approx 100\%$ execute, $+1.0$ utility over always-execute

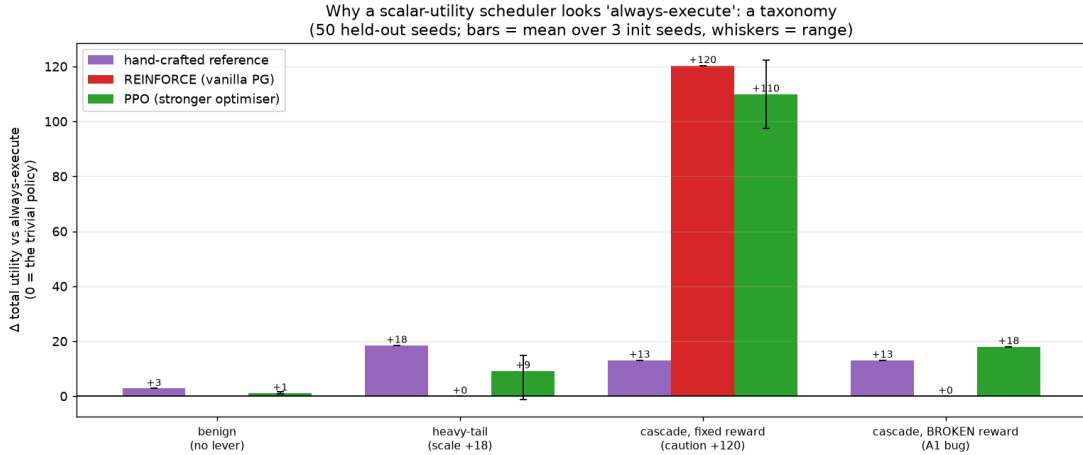


Figure 2: Why a scalar-utility scheduler looks “always-execute”: a taxonomy. Bars are Δ utility vs. always-execute (the 0 line is the trivial policy), mean over three initialisation seeds, whiskers the range, on 50 held-out seeds. REINFORCE (red) sits at ≈ 0 wherever the lever is a modest scaling improvement (**tight**, **heavytail**) and rises only at the enormous caution lever (**cascade**); PPO (green) rises wherever a lever exists and the reward is correct, and correctly stays near 0 on the benign environment.

Table 8: Lever check: a hand-crafted reference policy vs. always-execute on each environment (50 seeds, true metric). A positive significant Δ means a non-execute action pays.

Environment	always-exec	reference	Δ (p)
benign	111.5	114.3	+2.8 (0.06)
tight	82.8	89.1	+6.2 (0.03)
heavytail	451.6	470.0	+18.5 (0.03)
cascade	21.0	34.1	+13.1 (0.01)

— statistically above zero but operationally identical to the trivial policy, and well below the +6 to +28 it achieves where scaling pays. Crucially, PPO *declines* to scale here even though it scales aggressively elsewhere: it is not biased toward any action, it adapts to the environment.

(ii) **Optimiser, not reward.** On **tight** (scaling worth +6.2, $p = 0.03$), REINFORCE is stuck at 100% execute on all three seeds ($\Delta = 0.0$), while PPO learns to scale on all three and beats always-execute by +21 to +28 utility ($p < 10^{-7}$), lifting completion from 0.42 to 0.58. On **heavytail** (scaling worth +18.5, with whales worth 50 each), REINFORCE is again stuck on all three seeds, and PPO escapes on two of three (+14 to +15, $p < 10^{-5}$); the third is defeated by the high return variance the whales induce. We confirmed this is a genuine optimisation failure and not a discount artifact: on **tight** the scaling reference beats always-execute on the discounted training return as well (+2.06, $p = 0.01$), so REINFORCE’s own objective prefers scaling, yet it does not find

it. Under the *identical* reward and environment, a stronger optimiser removes the attractor.

(iii) **The reward bug is material.** On **cascade**, where overload triggers cascading failures, the corrected reward turns REINFORCE into a careful policy: it learns to scale up to hold load below the failure threshold, cutting the failure rate from 51% to 5% and lifting utility by +120 ($p = 1.8 \times 10^{-15}$). With the original (buggy) reward (**cascade-broken**), the same REINFORCE stays byte-identical to reckless always-execute (51% failure), and even PPO recovers only +18 — from the *indirect* value signal (failed jobs forfeit value), not from the missing failure penalty. The reward fix matters most for the weaker optimiser ($0 \rightarrow +120$) and substantially for the stronger one (+18 $\rightarrow$ +122).

(iv) **REINFORCE’s threshold.** REINFORCE escapes always-execute only when the lever is enormous (**cascade**, +120); it fails the modest scaling levers (+6, +18) on all six seeds. PPO escapes whenever a lever exists and the reward is correct (5/6 scaling seeds), and abstains correctly where none exists. The determinant of “always-execute” is the environment, the optimiser, or a reward bug — in no cell is it the reward’s linear shape.

6.6. Behavioural Summary

Inspection of the learned policies confirms the table. The benign and stuck policies emit **Execute** on 100% of steps. The escaping PPO policies on **tight/heavytail** emit **Scale_Up** on 7–11% of steps — concentrated in capacity-blocked states —

Table 9: The taxonomy. Utility on the 50-seed held-out pool (true metric); Δ is vs. always-execute. “A-E util” is the always-execute utility; each learner column gives Δ for three initialisation seeds. **Bold** marks a significant escape from always-execute ($p < 0.05$).

Environment	A-E util	ref. Δ	REINFORCE Δ (3 seeds)	PPO Δ (3 seeds)	Determinant
benign (no lever)	111.5	+2.8	— (trivial by constr.)	+0.8 to +1.5; $\approx 100\%$ exec	Environment: optimum trivial; PPO correctly does not scale
tight (scale, +6)	82.8	+6.2	+0.0 (all 3, stuck)	+21 to +28 (3/3, $p < 10^{-7}$)	Optimiser: REINFORCE too weak; PPO escapes 3/3
heavytail (scale, +18)	451.6	+18.5	+0.0 (all 3, stuck)	+14 to +15 (2/3; seed 7: -1.3)	Optimiser: PPO escapes 2/3; whale variance defeats the 3rd
cascade, fixed reward (caution, +120)	21.0	+13.1	+120 (51% $\rightarrow$ 5% fail)	+98 to +122	Adequate reward + lever: both learn caution
cascade-broken (A1 bug)	21.0	+13.1	+0.0 (reckless, 51% fail)	+18.0 (partial)	Reward bug: removes the direct failure signal

and otherwise execute. The caution-learning policies on **cascade** emit **Scale_Up** to keep CPU load below the cascade threshold, a single emergent rule that simultaneously raises throughput and avoids failures. No policy uses **Reprioritize_Queue** or **Pause_LowPriority_Job**, consistent with those actions being net-negative in every environment we tested.

7. Discussion

The retracted claim. The earlier framing — that the reward’s shape, not the algorithm and not the observation, determines the always-execute attractor — is false in every cell of Table 9. The invariances that motivated it (to reward weights, observation dimension, and seed) are real but uninformative, because they leave fixed the two factors that actually move the outcome: the environment

and the optimiser. The honest contribution is the taxonomy: a scalar-utility scheduler looks trivial when the environment has a trivial optimum, when the optimiser is too weak to find a real but modest improvement, or when the reward is mis-specified.

Why a CMDP is not “the corrective.” An earlier draft proposed reformulating the scalarised utility as a constrained MDP with learned dual multipliers as “the corrective the mathematics implies.” Table 9 shows this is mistargeted. On the benign environment a CMDP cannot manufacture a non-trivial optimum, because none exists — there is no caution lever for a constraint to bind on. On **tight**, **heavytail**, and **cascade**, the lever is real, and the corrective that actually works is a stronger *optimiser* (PPO) or the *corrected scalar reward* — neither of which is a CMDP. The CMDP machinery

may still be valuable for guaranteeing hard cost or failure budgets, but it is not the explanation for, or the fix to, the trivial attractor.

Implications for practice. Three lessons generalise beyond this simulator. First, before concluding that a scalar reward is to blame for a trivial learned policy, verify that the environment actually contains states where a non-trivial action pays; if it does not, the learner is correct. Second, a deep-RL “failure to beat a simple baseline” should be tested against a stronger optimiser before it is attributed to the objective: a modest, delayed improvement can be invisible to a high-variance estimator (REINFORCE) and obvious to a low-variance one (PPO). Third, verify the reward identity — that the cumulative per-step reward equals the reported metric — as a unit test; a telescoping defect on one term can silently delete an entire objective.

Reframing the original “negative result.” What survives is more useful than the original claim. We document a clean case in which an apparent “RL collapses to a trivial policy” result decomposes into a correct learner on a trivial environment, a weak optimiser on a non-trivial one, and a reward bug — and in which a competent learner, given a lever and a correct reward, adapts. That a careful, minimally confounded study of a scalar-utility scheduler can be fooled in three distinct ways is a more instructive message for the systems-RL community than a single “reward-shape failure mode.”

8. Threats to Validity

Simulator fidelity. The environment is a discrete-time single-cluster abstraction; it omits JVM warmup, executor motion, network and disk I/O, and straggler effects present in production Spark/Kubernetes systems. The taxonomy is a statement about the gradient landscape of a scalar utility under controlled environment edits; whether the specific operational consequences transfer to a production system is a real-system validation we do not claim.

Optimiser scope. We test REINFORCE and PPO. The claim “a stronger optimiser escapes the attractor” is demonstrated for PPO; we do not claim every algorithm escapes, and PPO itself fails one of three `heavytail` seeds. The honest statement is that the attractor is *not* algorithm-independent, which is the opposite of the earlier draft’s assertion.

Environment coverage. The five environments are minimal edits chosen to isolate single levers

(scaling, caution, value heterogeneity). They are not a random sample of production workloads; a heavy-tailed, capacity-exceeding, arrival-driven workload would exercise more levers simultaneously and is the natural next step.

Heavy-tail variance. On `heavytail`, the whale-induced return variance defeats PPO on one seed; this is a property of value-function credit assignment under heavy-tailed rewards and suggests reward normalisation or distributional critics as remedies, which we do not evaluate.

9. Reproducibility

All results trace to committed scripts and CSV artifacts. Each training and evaluation run writes a resolved configuration, a manifest (commit hash, config hash, seed list, library versions, wall-clock), and per-seed metrics. Seeds are partitioned into disjoint pools enforced at configuration load, and every new experiment’s pool and prediction were registered before running. The diagnostics (§5) — the reward-identity check, the local-optimality probe, the power/rliable analysis — and the five environment configurations are generated by committed scripts; the full taxonomy (Table 9, Figure 2) regenerates from a single evaluation script and its CSV. The complete test suite (140 tests) passes on a pinned environment. Code and artifacts are released [37].

10. Conclusion

A self-learning agent maximising a scalar utility for data-pipeline orchestration converges to a trivial always-execute policy. The tempting explanation — that the reward’s shape causes it — is false. We replace it with a measured taxonomy: always-execute appears because the environment’s optimum is genuinely trivial, because the optimiser (REINFORCE) cannot find a real but modest improvement that a stronger optimiser (PPO) finds easily, or because a reward-specification defect deletes the failure penalty. We further correct an underpowered equivalence claim, showing the learned policy is significantly if slightly worse than a trivial reflex baseline. A competent learner, given an environment with a lever and a correctly specified reward, adapts — scaling where scaling pays, throttling where caution pays, and abstaining where neither does. The lesson for systems reinforcement learning is to attribute a trivial learned policy to the environment, the optimiser, or the reward identity — checked individually — before attributing it to the shape of the objective.

References

- [1] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, “Resource management with deep reinforcement learning,” in *Proceedings of the 15th ACM Workshop on Hot Topics in Networks (HotNets)*, Atlanta, GA, USA, Nov. 2016, pp. 50–56. DOI: [10.1145/3005745.3005750](https://doi.org/10.1145/3005745.3005750)
- [2] H. Mao, M. Schwarzkopf, S. B. Venkatakrishnan, Z. Meng, and M. Alizadeh, “Learning scheduling algorithms for data processing clusters,” in *Proceedings of the ACM Special Interest Group on Data Communication (SIGCOMM)*, Beijing, China, Aug. 2019, pp. 270–288. DOI: [10.1145/3341302.3342080](https://doi.org/10.1145/3341302.3342080)
- [3] H. Mao, S. B. Venkatakrishnan, M. Schwarzkopf, and M. Alizadeh, “Variance reduction for reinforcement learning in input-driven environments,” in *Proc. 7th Int. Conf. Learning Representations (ICLR)*, New Orleans, LA, USA, 2019.
- [4] O. Alipourfard, H. H. Liu, J. Chen, S. Venkataraman, M. Yu, and M. Zhang, “CherryPick: Adaptively unearthing the best cloud configurations for big data analytics,” in *Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, Boston, MA, USA, Mar. 2017, pp. 469–482.
- [5] E. Cortez, A. Bonde, A. Muzio, M. Russinovich, M. Fontoura, and R. Bianchini, “Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms,” in *Proceedings of the 26th ACM Symposium on Operating Systems Principles (SOSP)*, Shanghai, China, Oct. 2017, pp. 153–167.
- [6] D. Narayanan, K. Santhanam, F. Kazhamiaka, A. Phanishayee, and M. Zaharia, “Heterogeneity-aware cluster scheduling policies for deep learning workloads,” in *Proc. 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Virtual, 2020, pp. 481–498.
- [7] A. Qiao et al., “Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning,” in *Proc. 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2021, pp. 1–18.
- [8] S. J. Subramanya, D. Arfeen, S. Lin, A. Qiao, Z. Jia, and G. R. Ganger, “Sia: Heterogeneity-aware, goodput-optimized ml-cluster scheduling,” in *Proc. 29th ACM Symposium on Operating Systems Principles (SOSP)*, Koblenz, Germany, 2023.
- [9] P. Zheng, R. Pan, T. Khan, S. Venkataraman, and A. Akella, “Shockwave: Fair and efficient cluster scheduling for dynamic adaptation in machine learning,” in *Proc. 20th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2023, pp. 703–723.
- [10] W. Kwon et al., “Efficient memory management for large language model serving with PagedAttention,” in *Proc. 29th ACM Symposium on Operating Systems Principles (SOSP)*, Koblenz, Germany, 2023.
- [11] A. Agrawal et al., “Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve,” in *Proc. 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Santa Clara, CA, USA, 2024.
- [12] Y. Zhong et al., “DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving,” in *Proc. 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, Santa Clara, CA, USA, 2024.
- [13] H. Mania, A. Guy, and B. Recht, “Simple random search provides a competitive approach to reinforcement learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- [14] A. Rajeswaran, K. Lowrey, E. Todorov, and S. Kakade, “Towards generalization and simplicity in continuous control,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [15] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, “Deep reinforcement learning that matters,” in *Proc. 32nd AAAI Conference on Artificial Intelligence*, 2018.
- [16] L. Engstrom et al., “Implementation matters in deep policy gradients: A case study on PPO and TRPO,” in *Proc. 8th International Conference on Learning Representations (ICLR)*, 2020.
- [17] M. Andrychowicz et al., “What matters in on-policy reinforcement learning? A large-scale empirical study,” in *Proc. 9th International Conference on Learning Representations (ICLR)*, 2021.

- [18] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, “Deep reinforcement learning at the edge of the statistical precipice,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.
- [19] S. M. Jordan, Y. Chandak, D. Cohen, M. Zhang, and P. S. Thomas, “Evaluating the performance of reinforcement learning algorithms,” in *Proc. 37th International Conference on Machine Learning (ICML)*, 2020.
- [20] A. Y. Ng, D. Harada, and S. Russell, “Policy invariance under reward transformations: Theory and application to reward shaping,” in *Proc. 16th International Conference on Machine Learning (ICML)*, 1999, pp. 278–287.
- [21] D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman, and D. Mané, *Concrete problems in AI safety*, arXiv:1606.06565, 2016.
- [22] J. Skalse, N. H. R. Howe, D. Krashenninikov, and D. Krueger, “Defining and characterizing reward hacking,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [23] A. Pan, K. Bhatia, and J. Steinhardt, “The effects of reward misspecification: Mapping and mitigating misaligned models,” in *Proc. 10th International Conference on Learning Representations (ICLR)*, 2022.
- [24] W. B. Knox, A. Allievi, H. Banzhaf, F. Schmitt, and P. Stone, “Reward (Mis)design for autonomous driving,” *Artificial Intelligence*, vol. 316, p. 103 829, 2023.
- [25] D. M. Roijers, P. Vamplew, S. Whiteson, and R. Dazeley, “A survey of multi-objective sequential decision-making,” *Journal of Artificial Intelligence Research*, vol. 48, pp. 67–113, 2013.
- [26] P. Vamplew et al., “Scalar reward is not enough: A response to Silver, Singh, Precup and Sutton (2021),” *Autonomous Agents and Multi-Agent Systems*, vol. 36, no. 2, p. 41, 2022.
- [27] E. Altman, *Constrained Markov Decision Processes*. Chapman and Hall/CRC, 1999.
- [28] C. Tessler, D. J. Mankowitz, and S. Mannor, “Reward constrained policy optimization,” in *Proc. 7th International Conference on Learning Representations (ICLR)*, 2019.
- [29] A. Stooke, J. Achiam, and P. Abbeel, “Responsive safety in reinforcement learning by PID Lagrangian methods,” in *Proc. 37th International Conference on Machine Learning (ICML)*, 2020.
- [30] J. Achiam, D. Held, A. Tamar, and P. Abbeel, “Constrained policy optimization,” in *Proc. 34th Int. Conf. Machine Learning (ICML)*, Sydney, Australia, 2017, pp. 22–31.
- [31] Y. Chow, O. Nachum, E. Duenéz-Guzmán, and M. Ghavamzadeh, “A Lyapunov-based approach to safe reinforcement learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, Montréal, Canada, 2018.
- [32] T.-Y. Yang, J. Rosca, K. Narasimhan, and P. J. Ramadge, “Projection-based constrained policy optimization,” in *Proc. 8th International Conference on Learning Representations (ICLR)*, 2020.
- [33] Y. Liu, J. Ding, and X. Liu, “IPO: Interior-point policy optimization under constraints,” in *Proc. AAAI Conference on Artificial Intelligence*, 2020.
- [34] R. J. Williams, “Simple statistical gradient-following algorithms for connectionist reinforcement learning,” *Machine Learning*, vol. 8, no. 3–4, pp. 229–256, 1992.
- [35] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” arXiv:1707.06347, 2017. arXiv: [1707.06347](https://arxiv.org/abs/1707.06347).
- [36] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” *International Conference on Learning Representations (ICLR)*, 2016.
- [37] A. M. M. Mohamed, *Research code for adaptive data pipeline orchestration*, GitHub repository, 2026. Accessed: Mar. 10, 2026. [Online]. Available: <https://github.com/ahmedmshazly/adaptive-pipeline-orchestration>